

Robert Brown rmb369
James Montague gjm235
Owen Sharp ods22

MAE 3780 Final Report

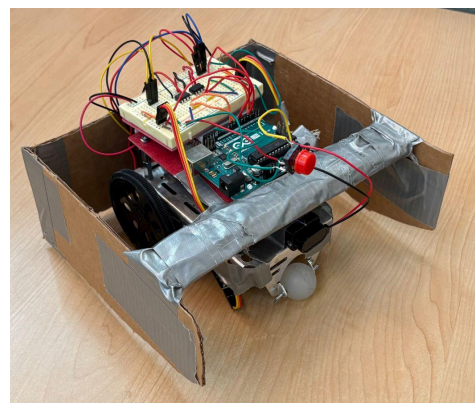
1. Robot Design & Strategy Overview

Our final robot was designed around a simple and reliable collection strategy rather than a complex transforming mechanism. Mechanically, the robot used the standard Boe Bot chassis with two rear drive wheels and a front roller ball. Around this base, we designed a lightweight passive collector to be printed, although due to time constraints we instead used cardboard. This collector formed a three-sided enclosure around the back and sides of the robot so that cubes entering the robot's path would remain trapped within its perimeter rather than being pushed away. A crossbeam was also added across the front upper portion of the robot to help support the structure and improve stiffness, particularly when having to use cardboard in the final design. This design was intentionally simple, lightweight, and easy to troubleshoot, which helped reduce the risk of mechanical failure during competition.

Electrically, the robot was controlled by an Arduino Uno mounted on top of the chassis and connected through breadboarded circuitry to the drive system and sensors. The robot used two QTI edge sensors mounted underneath the front underside of the chassis, one on each side, to detect the black border of the board and prevent the robot from leaving the field. A color sensor was also mounted near the center of the robot, although it was not ultimately used in the final competition strategy. The final electrical system was therefore centered mainly on reliable drive control and edge detection rather than more advanced sensing. The circuit diagram is provided in Appendix B.

In software, the robot used a hybrid strategy consisting of a hard-coded opening sequence followed by an adaptive roaming mode. At the start of a match, the robot faced forward and executed a programmed path designed to reach the middle region of the board as quickly as possible. It first drove to just short of the center, turned right, and swept toward the right edge. It then turned and crossed back through the middle region, swept toward the left side, and performed one final sweep before transitioning into roaming behavior. This opening path was intended to collect a large number of cubes early in the match, when the cube field was still dense and before other robots could interfere. After the initial sequence finished, the robot entered an adaptive roam mode in which it continued moving around the field while using the edge sensors to detect the border, reverse, and turn back into play.

Overall, the robot's operation relied on the interaction between a passive but effective collector and a conservative software strategy. Rather than attempting a complicated active intake or expanding structure, the robot used its walls to gather and retain cubes while following a path that repeatedly crossed the most cube-dense region of the board. This approach gave the robot a strong baseline performance, minimized malfunctions, and allowed it to perform consistently throughout competition.



2. Design Process Reflection

Our design process changed significantly over the course of the project. Early on, we considered a more elaborate concept involving a transforming collection mechanism that would deploy after the match began. The goal was to maximize collection width while still satisfying the starting size constraint. As the project progressed, however, it became clear that this approach would add mechanical complexity, require more fabrication time, and create more opportunities for failure. We therefore shifted our design philosophy away from mechanism complexity and toward reliability and repeatability.

This change was driven largely by prototyping and milestone testing. As we worked through the mobility and sensing requirements, we found that a robot with dependable drive performance and consistent edge detection already had a strong foundation for competition. More complicated collection ideas required tighter alignment, more moving parts, and more tuning than we could complete confidently within the project timeline. Rather than pursue a design with a higher ceiling but a lower chance of functioning reliably, we simplified the robot around a passive collector and a controlled drive path. This let us focus on the design features that most directly affected performance: staying on the board, reaching the cube-dense middle region quickly, and retaining cubes once collected.

Our final robot reflected a series of practical compromises. We initially planned to fabricate a more refined collector, but due to time constraints we replaced that plan with a cardboard enclosure that could be built quickly, adjusted easily, and repaired between tests. A similar process occurred in software. We originally explored using additional sensing, including the color sensor, to make the robot more responsive to field conditions. In practice, however, the most reliable solution was a hard-coded opening sequence that placed the robot in productive areas of the board early, followed by a simpler adaptive roaming mode based on the QTI edge sensors. This strategy was easier to tune, easier to troubleshoot, and less sensitive to sensing errors than a more complex decision-based algorithm.

3. Competition Analysis

Overall, our robot excelled in competition as we performed well in the round robin portion and advanced to the playoffs, where we unfortunately lost in the first round. We consistently were able to get 5+ blocks per round, which allowed us to win most of our matches. Our simple design and action process allowed minimal malfunctions, while still collecting ample blocks.

Many aspects of our design were successful in accomplishing their required functions, such as the initial hard coded action set. This loop allowed us to quickly acquire a large amount of cubes, and was often mirrored by the opponents. In some matches, the opponents would encounter some problem that prevented them from acquiring blocks, and getting these blocks at the start won us those rounds. When both robots were functioning, however, our secondary loop came into play. It was interesting to see how this loop affected matches, as oftentimes we ran into robots that performed a hard coded sequence and then stopped. Our robot would continue to bounce around the map, and in multiple cases it ended up picking up more blocks or stealing blocks from the opponent. That being said, this wasn't always perfect as sometimes the robot would lose blocks while bouncing around.

The only unexpected problem we had on competition day was caused by the boards being elevated, as opposed to in the lab where they were flush with the table. When our robot would reach the border, it would turn and the edge of our walls would catch on the lip. Luckily, we were able to quickly solve this by cutting off the bottom corner of the wall, allowing the robot to function as needed.

In general, our robot's main strength was its simplicity, as this gave it a high floor, preventing malfunctions and allowing us to get 5+ blocks a round. This did end up being our downfall, however, as when we reached the playoffs we ran into better bots that had upgraded speed and block acquisition systems.

4. Conclusions

As alluded to above, our robot performed well in the competition, so most of our conclusions pertain to the strengths we perceived in our robot. That being said, there were definitely things we could improve on.

A major factor that helped us perform well was sticking to the basics. Oftentimes in competitions like these, it is easy to dream up complex systems that give a competitive advantage. This was seen a lot in the competition, such as a group that had an expanding barrier that dropped down. Systems like these are prone to failure as they introduce unneeded complexity, whereas a simple robot has less moving parts that can break. They also can oftentimes not perform the action they were intended to do. That being said, for the sake of learning and fun, building complex systems is great. It just depends on what your goals are.

One of the main weaknesses we had was speed. Other groups invested in systems that increased their speed. We saw some groups with gear ratios and other motor setups, which allowed them to quickly jump out and grab a large number of cubes. This was seen in the winning group, whose robot drove out at high speed, picked up cubes, and turned off. This was a main deciding factor, as it gave them a high base line of cubes, and groups such as ours with looping sequences struggled to catch up.

All in all, if we had to do the competition again, we would largely keep it the same. I would advise future students to have a similar mentality as us, and opt for a robot that does the task in the simplest way possible. That being said, if we were to improve our robot, we would look into increasing its speed.

Appendix A: Bill of Materials (BOM)**Table 1: BOM**

Item:	Type:	Cost (\$):	Quantity:	Total (\$):
Rear hitch	3D Print	3.14	1	3.14
Hitch connector	3D print	1.78	1	1.78
			Total cost:	\$4.72

Appendix C: CAD Files and Drawings

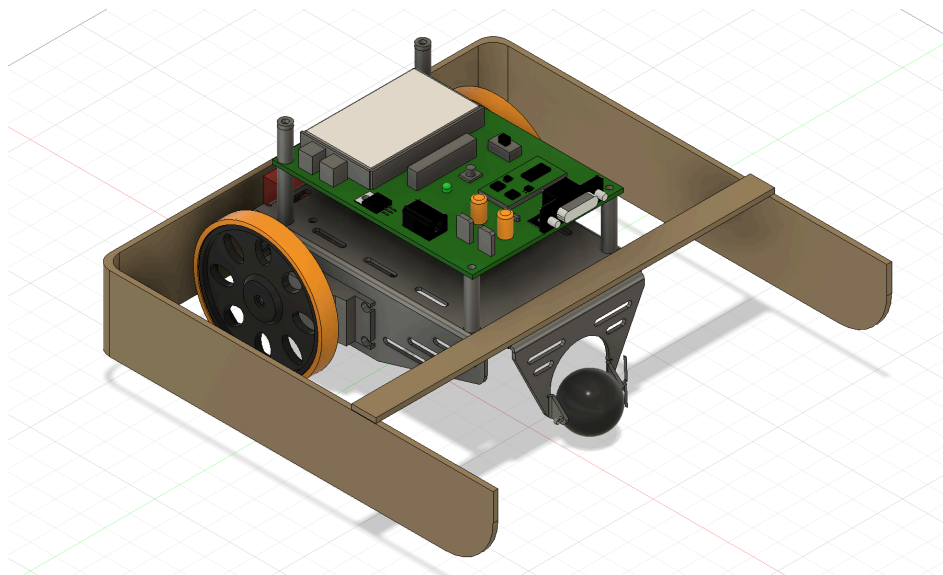


Figure 3: Final CAD of robot. Key additions include a cardboard exterior frame (brown), rubber bands on the wheels (Yellow), and the rear hitch and hitch connector (red).

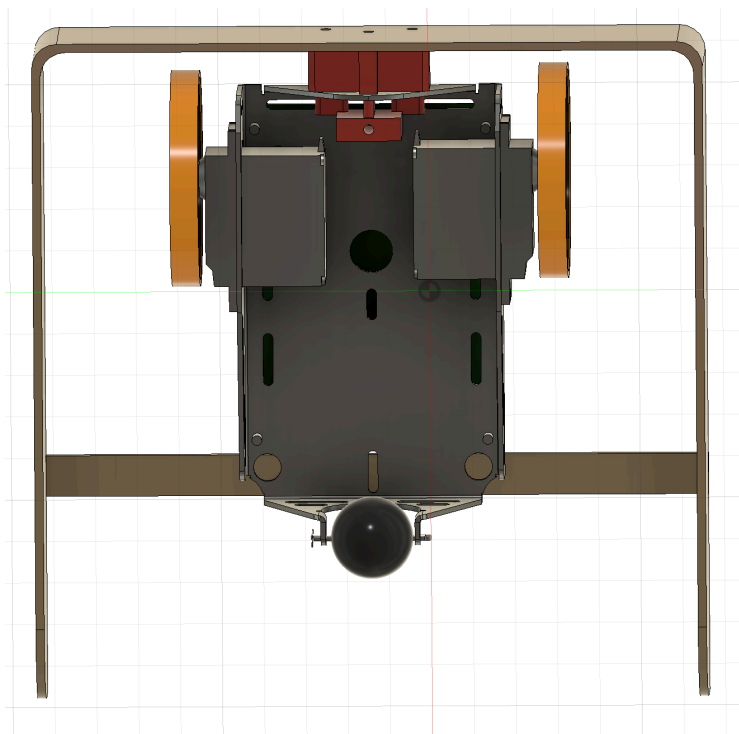


Figure 4: Bottom view of the robot. The rear hitch and hitch connector (red) are more easily seen, as well as the M3 screw attachment points for the cardboard frame (brown).

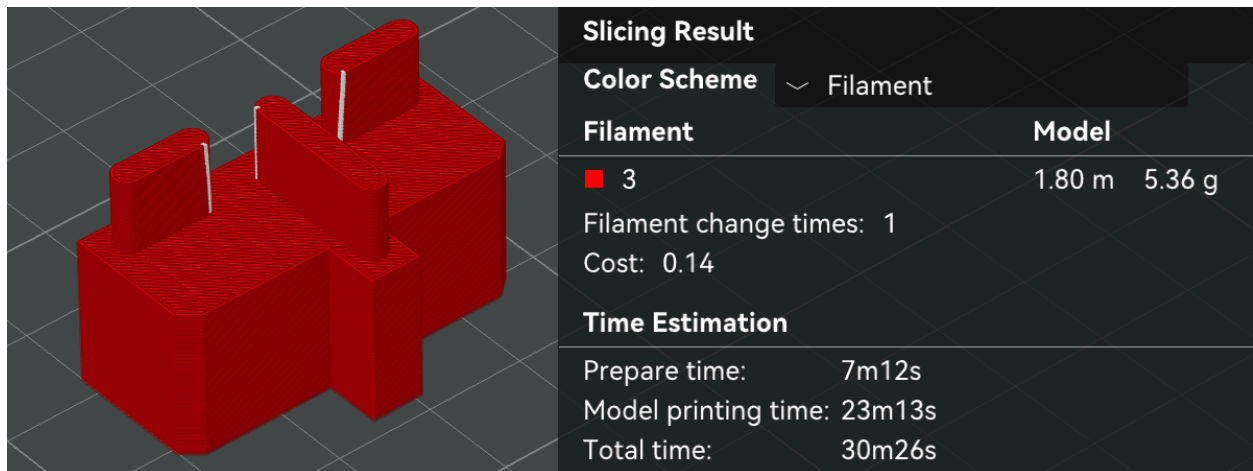


Figure 5: Slice of “rear hitch” using BambuStudio. Print mass was taken from the slice and used to calculate the part cost using $Cost (\$) = 1.00 + 0.40 * g = 3.14$ for PLA.

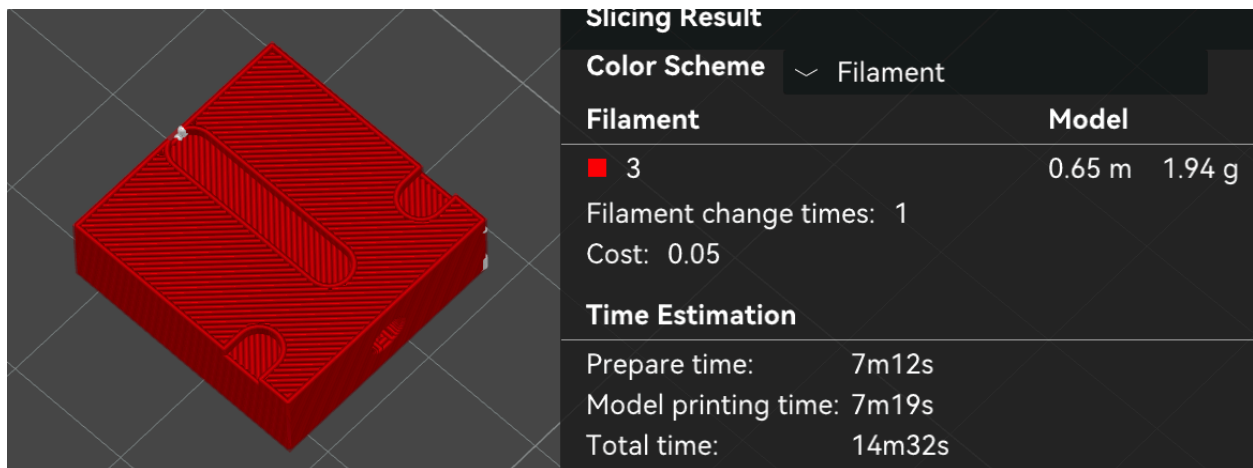


Figure 6: Slice of “hitch connector” using BambuStudio. Print mass was taken from the slice and used to calculate the part cost using $Cost (\$) = 1.00 + 0.40 * g = 1.78$ for PLA.

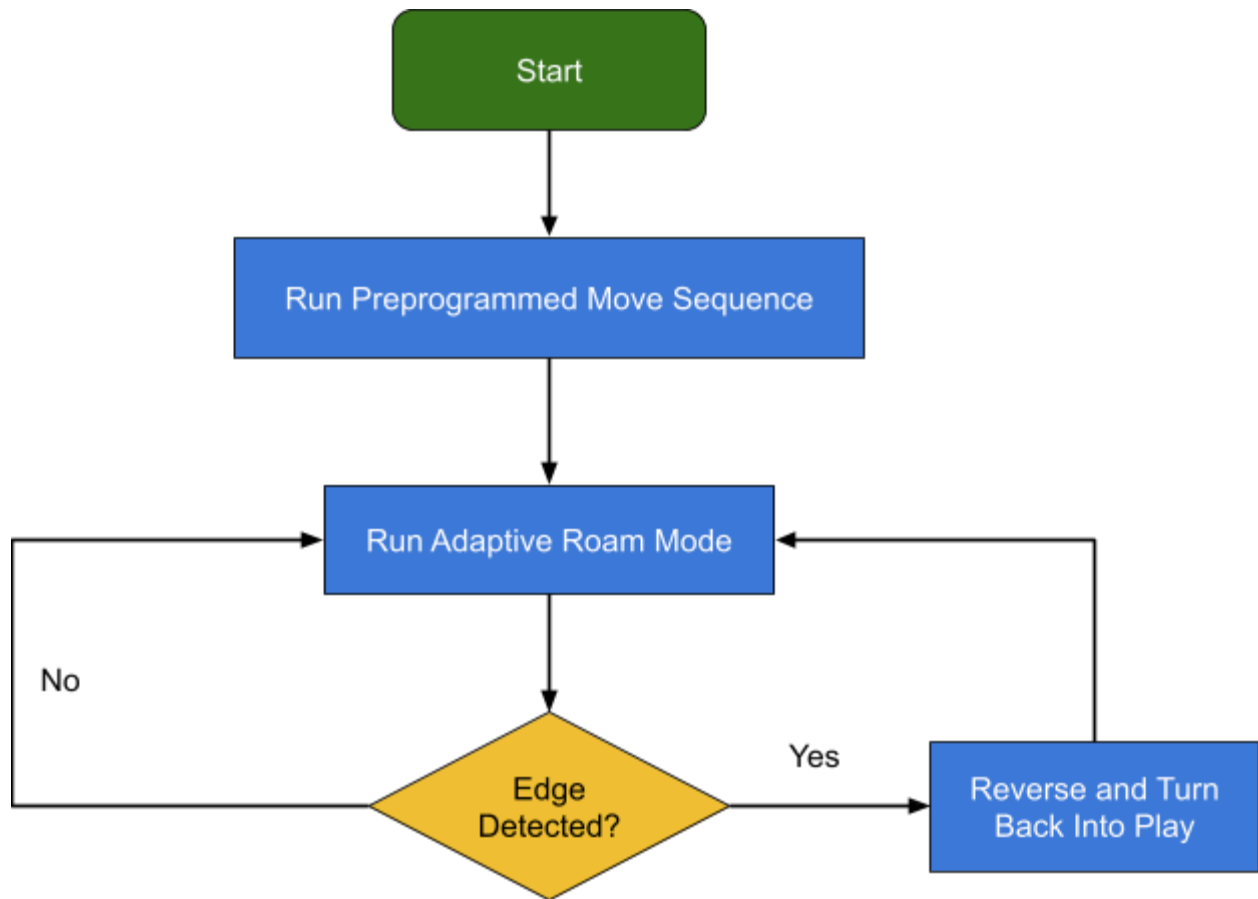
Appendix D: Flowchart

Figure 7: Flowchart depicting high-level logic used in the robot competition.

Appendix E: Code

```

#include <avr/io.h>
#include <util/delay.h>

// Edge sensor bit masks on port D
#define LEFT_SENSOR 0b00010000 // PD4
#define RIGHT_SENSOR 0b00001000 // PD3
#define BOTH_SENSORS 0b00011000

// Stop both drive motors
void stop_motors(void) {
    PORTB = 0b00000000;
    PORTD = 0b00000000;
}

// Variable millisecond delay
void wait_ms(unsigned int duration) {
    while (duration > 0) {
        _delay_ms(1);
        duration--;
    }
}

// Read left edge sensor
unsigned char left_edge_sensor(void) {
    return (PIND & LEFT_SENSOR);
}

// Read right edge sensor
unsigned char right_edge_sensor(void) {
    return (PIND & RIGHT_SENSOR);
}

// Check whether both edge sensors are active
unsigned char both_edge_sensors(void) {
    return ((PIND & BOTH_SENSORS) == BOTH_SENSORS);
}

// Short backward motion used during edge correction
void drive_backward_short(unsigned int duration) {
    PORTB = 0b00000001;
    PORTD = 0b01000000;

    wait_ms(duration);
}

```

```

    stop_motors();
}

// Simple timed right turn used during correction
void turn_right_simple(unsigned int duration) {
    PORTB = 0b00000001;
    PORTD = 0b10000000;

    wait_ms(duration);
    stop_motors();
}

// Simple timed left turn used during correction
void turn_left_simple(unsigned int duration) {
    PORTB = 0b00000010;
    PORTD = 0b01000000;

    wait_ms(duration);
    stop_motors();
}

// Edge recovery behavior during normal forward driving
void correct_if_edge(void) {
    if (both_edge_sensors()) {
        stop_motors();
        drive_backward_short(250);
        turn_right_simple(450);
    }
    else if (left_edge_sensor()) {
        stop_motors();
        drive_backward_short(100);
        turn_left_simple(180);
    }
    else if (right_edge_sensor()) {
        stop_motors();
        drive_backward_short(100);
        turn_right_simple(180);
    }
}

// Forward drive broken into short chunks so edge sensors can be checked repeatedly
void drive_forward(unsigned int duration) {
    while (duration > 0) {
        PORTB = 0b00000010;
        PORTD = 0b10000000;
    }
}

```

```

    if (duration >= 20) {
        wait_ms(20);
        duration -= 20;
    }
    else {
        wait_ms(duration);
        duration = 0;
    }

    stop_motors();
    correct_if_edge();
}

stop_motors();
}

// Short forward move helper
void drive_forward_short(void) {
    drive_forward(100);
}

// Longer backward move
void drive_backward(void) {
    PORTB = 0b00000001;
    PORTD = 0b01000000;

    wait_ms(700);
    stop_motors();
    wait_ms(1000);
}

// Timed right turn
void turn_right(unsigned int duration) {
    PORTB = 0b00000001;
    PORTD = 0b10000000;

    wait_ms(duration);
    stop_motors();
}

// Timed left turn
void turn_left(unsigned int duration) {
    PORTB = 0b00000010;
    PORTD = 0b01000000;
}

```

```
    wait_ms(duration);
    stop_motors();
}

// Preprogrammed opening path used at the start of the match
void run_sequence(void) {
    drive_forward(2300);
    wait_ms(20);

    turn_right(405);
    wait_ms(75);

    drive_forward(2000);
    wait_ms(20);

    turn_left(435);
    wait_ms(75);

    drive_forward(1200);
    wait_ms(20);

    turn_left(435);
    wait_ms(75);

    drive_forward(4000);
    wait_ms(20);

    turn_left(435);
    wait_ms(75);

    drive_forward(1200);
    wait_ms(20);

    turn_left(435);
    wait_ms(75);

    drive_forward(4000);
    wait_ms(20);

    turn_right(610);
    wait_ms(75);

    drive_forward(1000);
}
```

```

// Endless roaming behavior after the opening sequence finishes
void adaptive_roam(void) {
    while (1) {
        if (both_edge_sensors()) {
            stop_motors();
            drive_backward_short(250);
            turn_right_simple(600);
        }
        else if (left_edge_sensor()) {
            stop_motors();
            drive_backward_short(100);
            turn_left_simple(300);
        }
        else if (right_edge_sensor()) {
            stop_motors();
            drive_backward_short(100);
            turn_right_simple(300);
        }
        else {
            drive_forward(150);
        }
    }
}

int main(void) {
    // PD6 and PD7 are motor outputs
    // PD3 and PD4 remain inputs for the edge sensors
    DDRD = 0b11000000;

    // PB0 and PB1 are motor outputs
    DDRB = 0b00000011;

    // Make sure motors are off at startup
    stop_motors();

    // Automatically begin the programmed sequence, then roam forever
    run_sequence();
    adaptive_roam();

    return 0;
}

```